LiDAR Mapping and Surface Reconstruction from Two Dimensional Data on Rosmaster X3

Jacob Hudnut
Department of Computer Science
Rensselaer Polytechnic Institute
Troy, New York, USA

Elias Emons
Department of Computer Science
Rensselaer Polytechnic Institute
Troy, New York, USA

***Abstract*—Modern LiDAR systems are able to produce a vast quantity of data points in a very short timescale. Processing must be adequately inexpensive and efficient to keep pace with the rate of data production and compression such that storage is optimal. This project sought to gather a complete visual image from a collection of points in 2D space obtained by a LiDAR scan using the RPLiDAR A1 system on the Yahboom Rosmaster X3. Using point location data structures, and algorithms to cluster multiple points in space into representative line segments, LiDAR data can be significantly compressed. This reduces noise and data storage overhead, and the resulting line segments can be utilized to infer information about the LiDAR scene more easily than point cloud data in some cases.**

***Index Terms*—LiDAR, Surface Reconstruction, Surface Mapping, Data Compression.**

I. Introduction

Traditional implementations of Light detection and ranging (LiDAR) systems use an array capable of scanning in three dimensions to gather accurate visual data of some area or object, and use high accuracy surface reconstruction algorithms to create a three dimensional scene that can be more effectively parsed. For example, LiDAR is often used in satellites to process the shape and topology of poorly mapped areas on Earth. Not only can processing the LiDAR data allow for easier implementation of the data into usable applications, but it also can serve as a way to both de-noise the image and reduce the size of the data. Despite three dimensional LiDAR applications and relevant processing resources being readily available, applications and resources for two dimensional LiDAR systems are scarce. This relative difference in the number of resources between two and three dimensional LiDAR systems means that the use of lower cost, smaller two dimensional LiDAR arrays are often disincentivized. This project seeks to find an implementation to process two dimensional LiDAR point data.

II. Background

LiDAR is a technology similar to Sonar and Radar in that it utilizes the reflection of light waves from an emitted laser to obtain positional information of objects and the environment. For LiDAR data collection, a Slamtec RPLiDAR A1 mounted on a Rosmaster X3 was utilized. This system produces 8,000 sample points per second with one degree angular resolution, and a manufacturer stated maximum ranging distance of 12 meters. In order to implement this platform with the rest of this project, the LiDAR data is published by a ROS2 publisher node, and read from the topic by a subscriber to be exported as a .txt file. kD trees are an important data structure to segment and locate points and their neighbors, and as such are utilized in many implementations of LiDAR systems. A kD tree splits point data in half on one axis, recursively splitting the divided segments at the midpoint of the remaining data on an orthogonal axis, and it continues until a specific minimum points per segment threshold is reached.

LiDAR is a topic of significant interest in robotics research. Before beginning the project, several papers on the topic were reviewed. [Su, Bethel, and Hu] discussed the importance of segmenting LiDAR point cloud data, and introduced a novel segmentation technique focusing on octree decomposition to recursively divide the point cloud and represent it as a CAD model utilizing voxels and octants. Voxels are a representation of a value on a three dimensional regular grid, essentially a 3d version of a pixel. Octants are a division of a Euclidean three dimensional coordinate system that defines the eight regions based on the signs of the coordinates, similar to a 2d quadrant. Another element discussed in this paper was that much work done with LiDAR can be considered as 2.5d data, also known as a depth map. This means that the data can be represented as singular values for every point on a 2d plane. In the case of the two dimensional LiDAR data collected by the system discussed in this study, this data is similarly a depth map where singular values are mapped to points on a rotational axis, and as such may be considered 1.5d. The implications of this in terms of processing the data and reasoning about this could be motivation for future work on this topic. The segmentation technique in this paper consists of three steps. First, an octree decomposition, where point

cloud data is placed into octree bins, or voxels. The second is to split the points within the voxels using graph connectivity analysis with Tarjan's algorithm, and the third step is to recursively merge connected components across voxels until all voxels have merged to make a representation of the entire original point cloud. The strategy of modeling, which intentionally does not use k-nearest neighbors to save on compute time, was an interesting innovation. It could be further researched for its potential applications on the Rosmaster X3 and similar systems that utilize real time data processing. For 2d applications, pixels and quadrants could be utilized in place of voxels and octants.

Other methods for processing LiDAR point cloud data are similarly utilized, including some which take advantage of point cloud denoisers, as described in the paper [Tachella, Altmann, and Mellado]. Here, a point cloud denoiser algorithm is used to take input data from a three dimensional LiDAR scan and process the data points from target surfaces into a collection of two dimensional manifolds positioned in a three dimensional scene. The reconstruction algorithm, as a whole, is run on a Graphics Processing Unit, or GPU, and relies upon GPU memory to gather information of neighboring points. It also makes use of the parallel nature of GPU architecture to speed up. As for the LiDAR array that was used, a highly advanced pulsed laser was used to illuminate the scene, and a 32 by 32 pixel camera operating at a frame rate of 150 kilohertz was used to gather depth and intensity data from the laser illuminated scene. Overall, the algorithm described in the paper was able to operate with a runtime that increased from 10 milliseconds to 30 milliseconds as the number of LiDAR pixels increased from 64 by 64 to 400 by 400.

While multiple algorithms for processing LiDAR point data provide a reasonable solution, some approaches are particularly suited to their own use cases. As described by [Douillard et al.], there are three aspects to be considered with any given LiDAR segmentation algorithm. Firstly, data sparsity determined by LiDAR system and data gathering environment. Second, the ground representation. Some commonly utilized examples are grid based models, Gaussian Process models, or mesh based models. Finally, each algorithm is tested in multiple scenarios. The first method mentioned is the dense data segmentation pipeline with ground extraction, which uses a voxel grid to specifically segment dense data. The authors also discuss an algorithm for segmenting sparse data, Incremental Sample Consensus (INSAC). The INSAC algorithm and a variation on INSAC called Gaussian Process INSAC (GP-INSAC) were utilized along with the ground extraction scheme in the comparisons done by this paper. Through experimenting with various sets of LiDAR data, it was found that a "Cluster All" algorithm had the best overall performance in terms of simplicity, computation time, and accuracy.

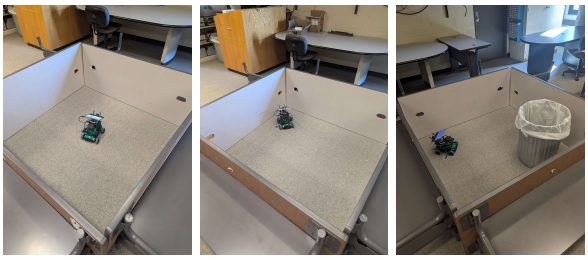
III. Motivation

The stated goal of the research project was to use some form of point location data structure to reconstruct a scene from LiDAR information. There are multiple implementations of point location data structures, such as Oct Trees, kD trees, and Binary Space partitions. The implementation in this project compared these methods to determine the most fitting point location structure for the data being processed. With oct trees, while implementation is the simplest, issues arise when querying point searches. Specifically, oct trees, despite their simplicity in construction, provide a less clear path in terms of finding nearest points, especially with CGAL. Neighbor search for Oct trees in the CGAL package implementation is minimal in comparison to other data structures. Binary space partitions remove the limitation of axis aligned segmentation. This may allow for construction of a more optimal structure in comparison to both kD trees and Oct trees, but the lack of axis alignment can lead to structures that are more difficult to reason about and more computationally expensive to determine partition placement. Given these considerations, kD trees were selected as a good middle ground between oct trees and binary space partitions in terms of simplicity, implementation, and performance.

IV. Methodology

Data was gathered in four sampling environments [figure 1]. Given the motivation of implementing surface reconstruction, a square sampling enclosure was built for the robot using classroom tables. These tables provided flat, linear surfaces on which the LiDAR system could be tested to see noise levels at differing ranges, and determine tuning parameters and the general viability of linear reconstruction on the point cloud data from the LiDAR system. Three of our point samples were taken from within the enclosure, one in the center, another in a corner of the enclosure, and another with a round trash can placed into the enclosure. In addition to the testing enclosure, data was also gathered from the floor of a busy classroom, with significantly more interruptions from chairs, desks, and human legs. This noisier dataset provided a more realistic sampling of what the robot might expect to encounter in real world usage, but would have been difficult to tune noise parameters around without the other, more controlled, datasets.

The data was collected from the Rosmaster X3, which uses ROS2 as a middleware to connect its sensing components, as a set of publisher and subscriber nodes. The A1 LiDAR system was opened using a node that publishes a raw text stream of angular data with headers

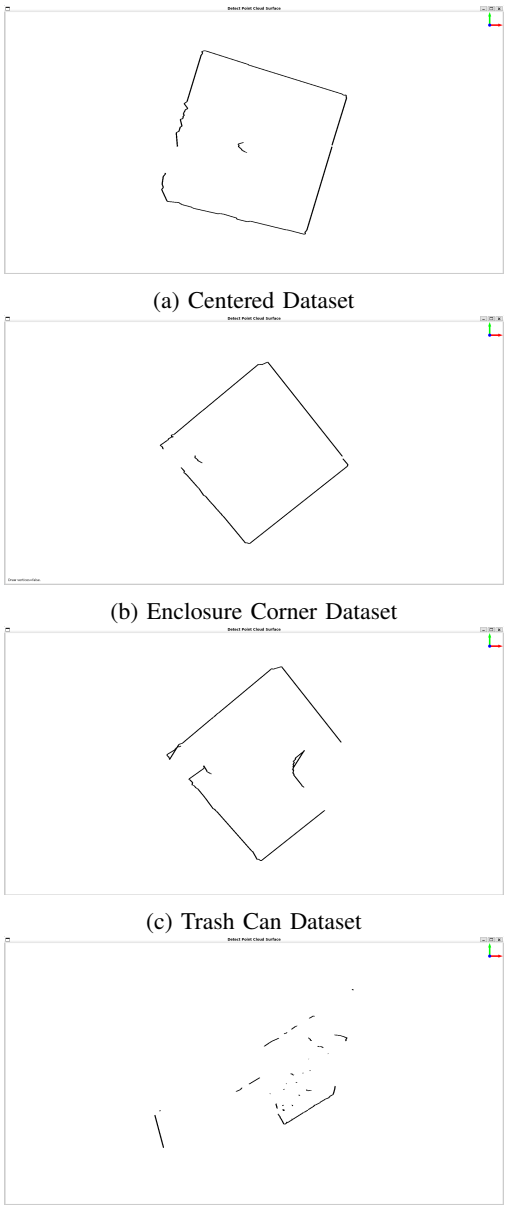


(a) Centered in Enclosure (b) Enclosure Corner (c) Enclosure with Trash Can

Figure 1: Enclosure Sampling Environments

containing information about read times, start and end angles, angle increments, and range dispersions, approximately at each 360 degree revolution of the laser. This text stream was recorded and fed into a .txt file to be processed by the surface reconstruction software. At this point, the authors diverged in approaches to line construction as to independently demonstrate multiple methods of surface reconstruction.

The initial approach utilized the CGAL Mesh package within a singular program to represent and display the output. Firstly, the program reads in the .txt file and converts the given data into a 2d point data structure, which is then used to construct a kD tree. In addition to this, each point is passed into a list to keep track of which points have yet to be processed. Following this, the list of unprocessed points, the kD tree, and a given point p are passed into an algorithm for calculation of edges. The algorithm begins by taking the given point p and finding its nearest neighbor, p(2), which is in the list of unprocessed points. From here, p is removed from the list of unprocessed points, and a search for p(2)'s nearest neighbor is queried. This process continues until some point, p(n), is found, which is in the unprocessed list but breaks certain linear parameters. The linear parameters are as follows: if p(n) is more than 25cm away from p(n-1), or if the angle formed by p, p(n-1), p(n) is less than 150 degrees and greater than 0 degrees, or more than -150 degrees and less than 0 degrees. If the first case is true, then a new edge from p to p(n-1) is created in the mesh, and the algorithm is recursively called with p(n) as the first given point. If the first case is false and the second case is true, then a new edge between p p(n-1) is created in the mesh, and the algorithm is recursively called with p(n-1) as the first given point. This algorithm continues until the list of unprocessed points hits a size of 0, or in other words, every point has been looked at and processed. The visualization of this approach is demonstrated in [Figure 2].



(a) Centered Dataset

(b) Enclosure Corner Dataset

(c) Trash Can Dataset

(d) Classroom Dataset

Figure 2: Naive Algorithm Visualization

Our second approach, based on a region growing algorithm, utilized a pipeline of multiple programs. Region growing is an iterative clustering technique that greedily groups points based on provided criteria. The sphere radius is provided for determining fuzzy sphere neighbors search using a CGAL Sphere neighbor query. This class creates a circle (in 2D case) or a sphere (in 3D case) centered at the query point with a user-specified sphere radius. All points, which belong to the sphere, will be treated as neighbors of the query point. The sphere radius utilized in this implementation was derived from data as 4 times the mean nearest neighbor distance. Another provided criteria is the max distance, which

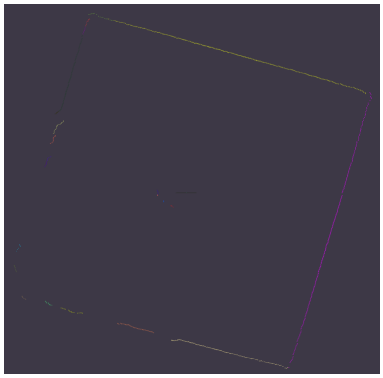
determines the maximum allowed distance from a given point to a line grouping before it is no longer considered a valid point on the line. This was also derived from mean nearest neighbor distance, and set to be twice this calculated value. A maximum angle parameter is also an argument to the region growing algorithm, but given that the surface normals for our data are constructs that carry no information from the original scene, this parameter was maximized to the largest allowed range to prevent it from not grouping points together due to these surface normals. The last parameter passed to the function was a minimum region size, which defines the number of points a region must contain before it is considered a line. This was set to 5. The region growing algorithm greedily selects points for its active region that fit the parameters, building a new region when no unprocessed points can be added to the active region. Each time a neighbor is added to a region, its neighbors are tested for the same region. The algorithm continues until all points have been processed into a region, or determined to not belong to a line. The process starts with a small C++ program that would take the first header and convert it into a .xyz file with dummy surface normals for use with the CGAL implementation of shape detection via region growing. The .xyz file was then passed to another C++ program that implemented the shape detection via region growing, specifically detecting lines on a point set. This program output the number of input points, the number of detected lines, and a .ply file that color coded the points from the LiDAR scan data based on their membership of these different line segments. The visualization of this approach is demonstrated in [Figure 3].

V. Evaluation

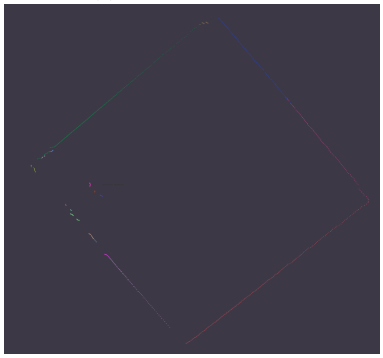
The four data sampling environments produced some significant differences in the results extracted from the code. The data itself was significantly different, even in number of input points. For example, in higher noise environments such as the classroom, the LiDAR system was more likely to report points at infinity, which were removed prior to line detection. The number of points was tracked, along with the number of produced lines. A compression factor was calculated by the following formula,

$$\text{Compression Factor} = \frac{\text{Detected Points}}{2 \times \text{Detected Lines}}$$

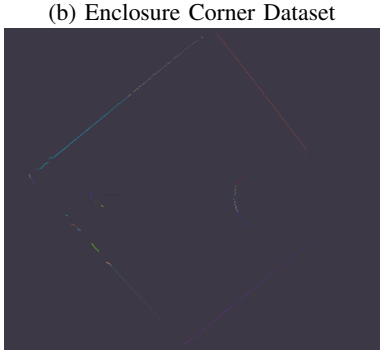
As each line segment can be represented as a pair of points, the compression factor represents the amount of memory that would be saved by representing the LiDAR data utilizing the detected lines, instead of storing the entirety of the point cloud. The following tables include the number of points for each dataset, the number of



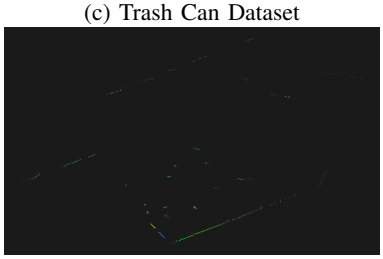
(a) Centered Dataset



(b) Enclosure Corner Dataset



(c) Trash Can Dataset



(d) Classroom Dataset

Figure 3: Region Growing Algorithm Visualization

lines detected by both line detection approaches, and the compression factor for each approach.

Table I: Compression Factor of Naive Approach on Multiple Datasets

Dataset	Points	Lines	Compression
Centered in Enclosure	938	102	4.6
Enclosure Corner	904	104	4.3
Enclosure with Trash Can	906	104	4.2
Classroom	880	208	2.2

Table II: Compression Factor of Region Growing Approach on Multiple Datasets

Dataset	Points	Lines	Compression
Centered in Enclosure	938	22	21.3
Enclosure Corner	904	18	25.2
Enclosure with Trash Can	906	23	19.7
Classroom	880	45	9.8

As demonstrated by these data, even a naive line detection algorithm can significantly reduce the storage space required to represent the information acquired from LiDAR systems. Even in a real-world data gathering scenario with significant noise, the implemented line detection algorithms were able to reduce stored data by a factor of 9.8, almost a decimal order of magnitude. In the context of mobile robot platforms using computers such as Jetson Nano and Raspberry Pi, limitations on memory may be significant, especially given the large quantity of data that can be produced by these LiDAR systems. For example, the system utilized for these tests, the Slamtec RPLiDAR A1, has a sampling rate of 8,000 samples per second. Assuming the point data were to be represented as single-precision floating point values on the cartesian plane, each point would require 8 bytes of data, so storing raw point clouds as a pair of floats has a memory cost of 64 Kilobytes per second. Most of the commercially available Rosmaster X3 systems have 8 Gigabytes (8,000,000 Kilobytes) of system memory, which, assuming the entirety of the memory is used for storing this point cloud data, leaves the Rosmaster x3 with enough memory to store approximately 35 hours of data. On certain applications, this may be sufficient, but for purposes such as surveying, a common use for LiDAR, having a compression scheme that requires approximately one tenth of the memory, as demonstrated by the Region Growing Approach, would allow the system to store approximately 350 hours of data, which could be a significant improvement for multiple use cases.

VI. Conclusion and Future Work

Overall, a region growing approach, such as the one included within the CGAL package, can be a useful tool for environment data representation from a 2D LiDAR scan. Not only did the compression factor significantly increase by a factor of 5 with this approach compared to the naive approach, but the visual output produced less oddities and errors. The region growing approach showed promising results. The compression rate was significant, and the output is machine readable. On top of this, the compression rate allows for less stringent memory requirements for any applications that seek to implement it. Possible future additions to these algorithms extend to areas of prediction, navigation, and reasoning. In terms of prediction and reasoning, it would be possible for data about out of range or concealed parts of the environment to be inferred upon based on existing data. For example, lines at the edge of the range of the LiDAR system can be extended outwards beyond the range as a form of prediction. In addition to this, using the second described approach, if co-circular points exist within the given data, a circular object that may be partially concealed to the LiDAR array can be inferred, and therefore represented in the program output. As for navigation, these programs could possibly be implemented to guide and work with the movement systems of the Rosmaster X3. In order for this to be possible, these programs would have to be optimized such that they can run in real time. If this is achieved, the processed data could be used as a map to guide the movement of the Rosmaster. Inertial measurement units from the Robot in combination with continuous scans during movement could allow for larger spaces to be mapped, in addition to allowing the Rosmaster to fully map objects. The current implementation leaves certain parts of the environment obscured due to the nature of LiDAR scanning.

References

[1] Tachella, J., Altmann, Y., Mellado, N. et al. Real-time 3D reconstruction from single-photon lidar data using plug-and-play point cloud denoisers. Nat Commun 10, 4984 2019. https://doi.org/10.1038/s41467-019-12943-7

[2] Yun-Ting Su, James Bethel, Shuowen Hu, Octree-based segmentation for terrestrial LiDAR point cloud data in industrial applications, ISPRS Journal of Photogrammetry and Remote Sensing, Volume 113, 2016, Pages 59-74, ISSN 0924-2716, https://doi.org/10.1016/j.isprsjprs.2016.01.001.

[3] B. Douillard et al., On the segmentation of 3D LIDAR point clouds, 2011 IEEE International Conference on Robotics and Automation, Shanghai, China, 2011, pp. 2798-2805, https://doi.org/10.1109/ICRA.2011.5979818.